microPAM_V3

Firmware for the LC-MARE passive acoustic recorder, built on the Adafruit Feather RP2040 Adalogger (one channel) or the Adafruit Feather RP2350 HSTX with 8 MB PSRAM (two channels). Either board can instead record all four ADC inputs at up to 192 kHz (`NCHAN` 4 in `config.h`). The acoustic front end is a TLV320ADC6140 ADC read over TDM, time is kept by an external RV-3028 RTC, and data are written to a microSD card.

This is a cleaned version of `LC-MARE/adalogger_V3` (Adalogger_V3a 3.1.0). Only two processing modes are kept, raw WAV and compressed raw; there is no DSP stage. Since firmware 3.3.0 the files and `config.txt` follow the microPAM_V5 format, so the V5 reader (copied to `Python/reader.py`) decodes them. Files of earlier V3 firmware use the old frame header and are not read by it.

The current version is 3.5.13. It has been tested on the RP2040 Adalogger (recording, configuration over serial, clock, reboot); the RP2350 and Teensy 4.1 builds compile but have not been run on hardware.

Pin connections

The firmware uses the following GPIO numbers of the RP2040 or RP2350. They are GPIO numbers, not Feather pin labels; on the Adalogger the SD card is on board, whereas the RP2350 HSTX Feather has no card slot, so that the card is wired through the base board.

Signal	Device	RP2040 Adalogger	RP2350 HSTX	Defined in
BCLK (TDM bit clock, output)	TLV320ADC6140	9	9	`src/rp2x.cxx` `I2S_BCLK`
FSYNC (TDM frame sync, output)	TLV320ADC6140	10	10	BCLK + 1 (PIO side-set)
DOUT (TDM data, input)	TLV320ADC6140	11	11	`src/rp2x.cxx` `I2S_DOUT`
ADC_EN (LDO enable, output)	ADC board	5	5	`src/adc.cxx`
SHDNZ (shutdown, active low)	TLV320ADC6140	6	6	`src/adc.cxx`
HP_ON (headphone amplifier, held low)	ADC board	14	14	`src/adc.cxx`
SDA (I2C, `Wire`)	ADC (0x4C), RV-3028 (0x52)	2	2	`src/rtc.h` `RTC_SDA`
SCL (I2C, `Wire`)	ADC (0x4C), RV-3028 (0x52)	3	3	`src/rtc.h` `RTC_SCL`
INT (alarm, active low, pull-up)	RV-3028	15	29	`src/global.h` `XRTC_INT_PIN`
SCK (SPI)	microSD	18 (`SPI1`)	22 (`SPI`)	board variant
MOSI (SPI)	microSD	19	23	board variant
MISO (SPI)	microSD	20	20	board variant
CS (SPI)	microSD	23	25	`src/global.h` `SD_CS`

Signal	Device	RP2040 Adalogger	RP2350 HSTX	Defined in
LED (flashes on card writes)	on board	13	7	`LED_BUILTIN`
NeoPixel (status)	on board	17	21	`PIN_NEOPIXEL`

The SPI pins are the defaults of the board variants in the `rp2040:rp2040` core (5.6.0): `SPI1` on the Adalogger is the card slot, `SPI` on the RP2350 is the Feather SCK/MO/MI header. The ADC address 0x4C corresponds to both ADDR pins low.

Build and upload

All scripts are PowerShell, live in `scripts/`, and use the arduino-cli named by the environment variable `ARDUINO_CLI`, else the one bundled with Arduino IDE 2 (`%LOCALAPPDATA%\Programs\Arduino IDE\...\arduino-cli.exe` or the same under `%ProgramFiles%`), else `arduino-cli` on the `PATH`, always with the data directory `%LOCALAPPDATA%\Arduino15`. Required: the `rp2040:rp2040` core (Earle Philhower, tested with 5.6.0), which brings the Adafruit NeoPixel library along. A script that needs arduino-cli stops with a message naming what is missing. Compiled output goes to `build/<target>/`.

Script	Action
`scripts\build.ps1 [rp2040|rp2350|all]`	compile only, no board needed
`scripts\flash_rp2040.ps1`, `flash_rp2350.ps1`	compile and upload
`scripts\upload_rp2040.ps1`, `upload_rp2350.ps1`	upload the existing `build/<target>/` firmware
`scripts\upload_uf2.ps1 <file.uf2> [-Port COMn]`	upload a ready-built firmware file; no compiler needed (see below)
`scripts\clean.ps1 [target]`	remove build output
`scripts\gen_intellisense.ps1 [target]`	regenerate `compile_commands.json` for VS Code
`scripts\workbench.ps1 [-Dev]`	start the workbench (see Workbench); also by double-click on `workbench.cmd` or `workbench_dev.cmd`
`scripts\setup.ps1 [-Force]`	create (or recreate) `Python/.venv`, report whether Arduino is installed; also by double-click on `setup.cmd`
`bash scripts/setup.sh [--force]`	the same on macOS (Python from python.org, or Homebrew `python` with `python-tk`)
`scripts\make_package.ps1 [-NoFirmware]`	build the user package `dist/microPAM_V3_<Version>.zip` (see User package)

The upload first looks for the BOOTSEL drive (`RPI-RP2` or `RP2350`) and copies the UF2 file onto it; otherwise it uploads through the serial port reported by arduino-cli. The same commands are available as VS Code tasks.

arduino-cli writes its memory report to stderr, which PowerShell may print as a `NativeCommandError`. This is not a failure; the result is given by `OK <target>` or `All builds passed`.

Uploading a ready-built firmware file

A `.uf2` file can be uploaded without Arduino IDE or compiler, by `scripts\upload_uf2.ps1`, by `Python\uf2_upload.py`, or in the workbench (Firmware tab, user mode). The file is first

checked: it must consist of valid UF2 blocks, built for one chip (RP2040 or RP2350). If the board is already in BOOTSEL mode, the file is copied to its boot drive; otherwise the board is reset into the boot loader through its serial port (1200-baud touch) and the boot drive is awaited. Only a drive carrying the boot ROM's `INFO_UF2.TXT` is written to, and a file built for the other chip is refused before anything is written.

To flash entirely by hand: disconnect the battery, connect USB, hold BOOT (the button furthest from the USB connector), press RESET briefly, release BOOT, and copy the `.uf2` file to the drive that appears.

Configuration

`config.h` holds the compile-time defaults: processing mode, duty cycle, sampling frequency, channel selection and the file metadata. `src/global.h` sets the hardware: the preamplifier type (`PREAMP`), and everything derived from the defaults (channel count, pins, queue size, MCU clock).

`NCHAN` selects the channels: 0 keeps the board default (RP2040: one channel, ADC input 2, or input 1 with `PREAMP` 2; RP2350: inputs 2 and 3), 4 records inputs 1 to 4 in this order and caps the sampling frequency at 192 kHz. The TDM bit clock is then 24.576 MHz, as for two channels at 384 kHz, and the MCU runs at 144 MHz above 96 kHz. Four channels on the RP2040 have only the 160 kB queue as buffer; check `missed: acq` in `ICMT`.

`PROC_MODE`	Output
0	raw 32-bit samples, `.wav`
1	compressed raw, `.bin` (512-byte WAV header followed by compressed frames)

At boot `/config.txt` on the SD card overrides the defaults (example in `data/config.txt`). One parameter per line, `!<key> <value>`, terminated by `#` or `;`:

Key	Parameter
`a`	file length (s)
`o`	on time per repetition period (min)
`r`	repetition period (min); continuous recording if `r` ≤ `o`
`f`	sampling frequency (kHz)
`g`	analog gain (dB)
`s c n p e l`	metadata ISRC, ICMS, IART, IPRD, ISBJ, INAM (single words); ISRC carries the calibration (see Calibration)
`1 2 3 4`	recording hours: record in hours h with `1` ≤ h < `2` and `3` ≤ h < `4` (end hours excluded since 3.4.2)
`m`	PROC_MODE; written by the firmware, ignored when read (compile-time)
`x`	delayed start time, `yyyy-mm-dd_hh:mm:ss`

The file is rewritten with the current values whenever the system stops or hibernates, and on the menu command `w`.

Serial menu

Command	Action
`s`	start acquisition
`e`	end acquisition
`p`	print system and acquisition parameters
`c`	check and correct the RTC (only when stopped)
`x n`	hibernate until n full hours ahead; `x 0` reboots
`y n`	hibernate until n full minutes ahead; `y 0` reboots
`b`	store configuration and reboot
`w`	store configuration (`/config.txt`) now
`?k`	print parameter `k` (`a o r f g m s c n k e l x 1 2 3 4`, `u` UID, `d` RTC time)
`!k v`	set parameter `k` to `v` (`a o r f g s c n k e l x 1 2 3 4`, `d` RTC time)

The menu keys equal the `config.txt` keys except that IPRD is `k` (the file's `p`) and `f` is in Hz (kHz in the file). `!d yyyy-mm-dd hh:mm:ss` sets the external and the MCU RTC. Changed parameters take effect at once, except the start time `x`, which is evaluated at boot; they reach `/config.txt` with `w`, `b`, a stop or a hibernation.

Boot and status LED

At boot the firmware reads the RTC interrupt pin. A low pin means that the board was woken by an RTC alarm; it then starts recording without waiting for USB. Otherwise it waits up to 5 s for a USB serial connection, and, once connected, a further 200 ms so that the host receives the first lines. After the RTC has been set up, the alarm flag of the RV-3028 is cleared and its alarm interrupt disabled, which releases the interrupt pin; a flag left from an earlier wake-up could otherwise make every later boot look like an alarm wake-up. The boot log reports this as `XRTC INT at boot: high|low; alarm flag 0|1 (cleared)`.

The MCU clock is set from the RV-3028, the SD card and `/config.txt` are read, and, if the start time `x` lies within the next year, the board hibernates until then. With an SD card present recording starts automatically; without one, the first words of each block are printed once per second.

NeoPixel	Meaning
red, then yellow	booting
white (brief)	opening a file
off	recording
blue	no SD card, or a file could not be opened
green	stopped

Time stamps

The board clock carries no time zone. The Unix seconds in each compressed frame are the board time read as UTC, and the file names and folders use the board time directly. For true

Unix time, set the board clock to UTC (workbench Device tab, option UTC); if it is set to local time, the stamps are local time, as in microPAM_V5.

Data format

Files are stored as `/<uid>_<yyyymmdd>/<hh>/<uid>_<yyyymmdd>_<hhmmss>.<wav|bin>`. Each file starts with a 512-byte WAV header whose LIST/INFO chunk carries the metadata. As in microPAM_V5, `IKEY` holds `uid; t_acq; t_on; t_rep; fs_kHz; gain; battery; shift; mode; nbuf_proc; nblks; navg; h_rec[0..3]; version` and `ICMT` holds `Version: …; MCU: …; missed: acq <n>`.

A compressed frame (microPAM_V5 record) encodes 1024 samples, all channels interleaved:

Word	Content
0	`0x434F4D50` (“COMP”)
1	Unix seconds
2	`micros()`, free-running µs counter
3	`nb`, bits per difference
4	`nd`, number of packed words
5 … 5+NCH−1	first sample of each channel
following `nd` words	sample differences, `nb` bits each, MSB first

The samples are shifted right by 12 bit before differencing. Each frame is zero padded to a multiple of 512 bytes, and the frames follow each other without gaps.

Workbench

The user guide to the workbench, with its configuration modes and the recording schedules, is `WORKBENCH.md`; this section summarizes it for developers.

`Python/workbench.py` is a local tool built from the microPAM_V5 Python tools (reader, browser, configurator, config format). Start it from Explorer by a double-click on `workbench.cmd` in the project folder, or `workbench_dev.cmd` for developer mode, or from a PowerShell prompt with

```
scripts\workbench.ps1 [-Dev]
```

The two `.cmd` files run the same script, independent of the PowerShell execution policy, and keep their window open with the error message if the start fails. On first use the script runs `scripts\setup.ps1`, which creates `Python/.venv` from `Python/requirements.txt`. It needs Python 3.10 or later with tkinter, found through the `py` launcher, `python` or `python3`; the Microsoft Store stub of `python` is skipped, as it only opens the Store. If the pinned versions have no wheels for the installed Python, the current numpy, matplotlib and pyserial are installed instead. If the installation fails, the incomplete venv is removed, so that the next start tries again. On macOS, `bash scripts/setup.sh` creates the venv in the same way (it passes over Apple’s Python 3.9 and requires Tk 8.6 or later), and `Python/.venv/bin/python Python/workbench.py` starts the workbench in user mode; developer mode needs the PowerShell scripts. The workbench has four tabs:

Tab	Function
Firmware	developer mode: select target and `PROC_MODE` (edits `config.h`), build, upload, build + upload, clean, through the scripts above; user mode: upload a ready-built `.uf2` file chosen on disk
Configuration	all `config.txt` parameters: defaults from `config.h`, load/save a local file, read from and write to the board over serial (written values are stored with `w`)
Device	serial connection, start/stop, reboot, hibernate, board clock (read, set to PC time), serial monitor
Data	browse local folders, header of a recording, time series and spectrogram (colour range adjustable in 5 dB steps), Welch spectrum, conversion of `.bin` files and folders to WAV

The Firmware tab starts in user mode. Developer mode is selected on the tab, or at start with `scripts\workbench.ps1 -Dev` (`workbench.py --dev`); it is not available when the source (`config.h`) is not present, or when arduino-cli or the `rp2040` core is not found (searched as by the scripts). In that case the tab names what is missing, and "Check again" enables developer mode once Arduino has been installed, without a restart. An upload releases the serial port first. After a successful upload, and after a reboot command (Reboot button, or `x0`, `y0`, `b` typed in the monitor), the workbench reconnects by itself, trying for up to 10 s. It finds the board by its USB serial number, so a changed COM port number does not matter, and as the board waits 5 s for USB at boot, the complete boot log appears in the monitor. After an upload the workbench switches to the Device tab as soon as the board is connected again, so that its output is in view; if the board does not return, the Firmware tab with the upload log stays open. Writing the configuration to the board sends every parameter except `m` (compile-time) and, by default, stores it on the card with `w`. The Reboot button sends `x 0`, which does not rewrite `/config.txt`. `Python/micropam_browser.py` also runs on its own as the V5 file browser.

Calibration

The source string `SRC_str` in `config.h` (`!s` in `config.txt`), which the firmware writes into the ISRC chunk of every file, carries the acoustic calibration in the form `<name>-<C0>`. C0 is the level in dB re 1 µPa that corresponds to one count of the 24-bit ADC word at 0 dB analog gain, so that it does not depend on the settings of a recording. One count of the samples of a file corresponds to

C = C0 − g + 20 log10(2^(SHIFT − 8)) dB re 1 µPa,

where g is the analog gain in dB and SHIFT the right shift, both taken from `IKEY`. A sample I is then a sound pressure I · 10^(C/20) µPa. The default `LC17-38.7` follows from the resistor measurement on unit 1B843C2E (`tests/characterization_1B843C2E.md`), whereby one count is 42.8 dB re 1 µPa at 20 dB gain and SHIFT 12, i.e. C0 = 42.8 − 20 − 24.1 = 38.7. `Python/micropam_calib.py` (`count_level_db(info)`, `to_upa(data, info)`, and the same module in pamlib as `pamlib.io.micropam_calib`) retrieves the calibration from a file header. With it, the Data tab shows the calibration in the header popup, the time series in Pa, and the spectrogram and the Welch spectrum in dB re 1 µPa²/Hz; without a numeric suffix the viewers stay in counts. A suffix outside −20 to 100 dB is not taken as a calibration, so that a hydrophone sensitivity in the older form `AS1-208` is not mistaken for one.

"To WAV…" on the Data tab (or `python Python/bin2wav.py DEST SRC …`) converts the selected `.bin` files, and all `.bin` files in selected folders and their subfolders, into a destination folder; a folder keeps its tree below it. The WAV file is what `PROC_MODE` 0 writes: the original header with `IKEY` set to `SHIFT` 0 and `PROC_MODE` 0, and 32-bit samples shifted back by 12 bits, so that full scale is 2^{31} and the dropped low bits are zero. Records missed at acquisition (`missed: acq` in `ICMT`) are filled with zeros, so that the WAV keeps the sample timing: the `micros()` stamps of successive records differ by one record length (1333.33 µs for 256 frames at 192 kHz) within about 2 %, and a larger difference gives the number of missing records. Legacy (pre-3.3.0) files carry only `millis()`, which is too coarse for this; their gaps stay closed.

User package

`scripts\make_package.ps1` builds `dist/microPAM_V3_<Version>.zip` for users, with the version taken from `src/global.h`. The zip holds a folder `microPAM_V3` (the sketch name) with the firmware source, `data/config.txt`, the workbench and its Python files, the scripts the workbench runs, `setup.cmd`, `workbench.cmd`, this README, the setup guide `SETUP.md` for Windows and macOS, the workbench user guide `WORKBENCH.md`, and a user `README.txt` (from `package/README.txt`, with version, date and commit filled in). This README, `SETUP.md` and `WORKBENCH.md` are also converted to PDF by pandoc with the Typst engine (both must be on the `PATH`, otherwise the PDF is left out with a warning); `package/readme_pdf.typ` lets the tables break across pages and sets them left-aligned in a smaller font. The tests, the VS Code files and `gen_intellisense.ps1` stay out. `firmware/` holds ready-built files for both targets in both modes, `microPAM_V3_<Version>_<target>_bin.uf2` (`PROC_MODE` 1) and `_wav.uf2` (`PROC_MODE` 0), compiled with the other defaults of `config.h` in a scratch copy, so that `config.h` is not touched; `-NoFirmware` leaves them out and needs no Arduino. The package is about 0.6 MB; the Python environment that `setup.cmd` creates from it is about 150 MB and needs an internet connection once.

The package folder should be unzipped to a short path (e.g. `C:\microPAM_V3`). Unzipped under a path of about 200 characters, the Pillow DLLs inside the venv passed the 260-character limit of Windows, and matplotlib failed to import; `setup.ps1` checks the imports and then points to the path length.

Source layout

File	Content
`microPAM_V3.ino`	setup and main loop (core 0), acquisition start (core 1)
`src/rp2x.*`	PIO TDM input, DMA, dormant sleep, UID, NeoPixel
`src/compress.*`	frame compression (PROC_MODE 1)
`src/queue.*`	block queue between acquisition and filing
`src/filing.*`	SD card, WAV header, file management, duty cycle, `config.txt`
`src/adc.*`	TLV320ADC6140 configuration
`src/rtc.*`	MCU RTC and RV-3028
`src/menu.*`	serial menu
`src/i2c.*`	I2C register access

File	Content
`Python/`	workbench, V5 reader, config format, file browser, UF2 uploader
`workbench.cmd`, `workbench_dev.cmd`, `setup.cmd`	double-click launchers for the workbench and the Python setup
`package/README.txt`, `package/readme_pdf.typ`	template of the user package's README; Typst style of `README.pdf`

Version history

Version	Change
3.2.0	cleaned from Adalogger_V3a 3.1.0: raw and compressed modes only; queue rewritten (raw mode recorded nothing before); fixes to gain setting, WAV sample rate, `config.txt` truncation, recording-hours alarm time
3.3.0	microPAM_V5 file and `config.txt` format; menu keys `? m ?x`, `?/!s c e`, `w`; `!d` also sets the MCU RTC
3.3.1	RV-3028 alarm flag cleared at boot
3.3.2	200 ms pause after USB connects, so the complete boot log reaches the host
3.4.0	`NCHAN` 4 in `config.h`: all four ADC inputs, sampling frequency up to 192 kHz
3.4.1	calibration in ISRC: `SRC_str` "LC17-38.7" (`<name>-<C0>`), used by the Data tab viewers
3.4.2	recording hours `!1`–`!4` exclude the end hours: record in [1, 2) and [3, 4), so that `0 6 20 24` records 00:00–06:00 and 20:00–24:00
3.5.0	Teensy 4.1 target: one TLV320ADC6140 on SAI1, 2 channels (4 with `NCHAN` 4) at up to 192 kHz, SD card in the built-in SDIO slot, SNVS RTC instead of the RV-3028, hibernation by power-off with the SNVS alarm; carrier board by `HW_BOARD`
3.5.1	Teensy 4.1: CPU at 24 MHz (build setting `speed=24`); SD card to stand-by by CMD7 deselect between block writes (`SD_CMD7` in `config.h`, after `notes/SD_interface.md`), `mdt` then includes the programming wait
3.5.2	Teensy 4.1: core from the 24 MHz crystal with the ARM PLL off, unused clock gates closed (`T41_POWER` 6); `PHY_OFF` (Ethernet PHY power-down, off by default, no saving measured); menu `z<n>` applies the power steps at run time, `z0` prints a clock report
3.5.3	Teensy 4.1: disk write unit (queue block) doubled to 64 kB (`MD` 16, IKEY reports it), queue 7 blocks of the same 448 kB; `ADC_PWR` 0 test switch (ADC board unpowered)
3.5.4	Teensy 4.1: queue in PSRAM, 128 blocks of 64 kB = 8 MB (about 10 s at 192 kHz, 2 channels, compressed), fallback to 448 kB in DMAMEM without PSRAM

Version	Change
3.5.5	Teensy 4.1: disk write unit 256 kB (`MD` 64), queue 32 blocks = 8 MB in PSRAM (fallback without PSRAM: 1 block in DMAMEM, not for deployment); files then close up to one block (about 0.3 s at 192 kHz, 2 channels, compressed) after the `t_acq` boundary
3.5.6	RP2350 with PSRAM: disk write unit 256 kB (`MD` 64), queue 28 blocks = 7 MB; RP: `SD_CMD7` also sends CMD7 (RCA 0) over SPI after each block, dropped if the card answers illegal command; test switch `NB_MIN` (config.h) sets a minimum bit width of the compressed differences (0: off)
3.5.7	RP2040 Adalogger: microSD over 4-bit PIO SDIO (SdFat `PioSdioCard`, GPIO 18-23), switch `SD_SDIO` (config.h, 1 by default; 0: SPI as before); no CMD7 with SDIO, `syncDevice()` only
3.5.8	A recording started after a stop no longer closes its first file after one block (the `t_acq` phase is now taken when a file opens); that early close had let the next file open in the same second, under the same name, and append to it
3.5.9	RP2040 PIO SDIO: with `SD_CMD7` 1 the card goes to stand-by by CMD7 after each block, as on the Teensy, if SdFat carries `patches/SdFat_PioSdio_cmd7.patch` (`scripts/patch_sdfat.ps1`, again after each rp2040 core update); without the patch `syncDevice()` only, as in 3.5.7; PIO SDIO clock `SD_SDIO_MHZ` (global.h) 24 MHz at any `CLK_MULT`, at most 25 MHz (high speed by CMD6 gives CRC errors with the PIO driver)
3.5.10	RP2040: sampling frequency up to 384 kHz (`FSAMP_MAX`, was 192 kHz); the 160 kB queue then holds only about 0.2 s of compressed data
3.5.11	RP2040: MCU clock 96 MHz also above 192 kHz (was 144 MHz); 192 MHz was tested at 384 kHz and gave the same losses during card stalls at about 15 mA more
3.5.12	RP2350: MCU clock 96 MHz at all sampling frequencies above 96 kHz (was 144 MHz at 384 kHz with 2 channels and at 192 kHz with `NCHAN` 4); not yet tested on the board
3.5.13	Each card cycle (wake, write, stand-by) writes up to `SD_DRAIN` (global.h, 8) queued blocks, so a backlog after a slow card write goes to the card as one longer write command (CMD25 continued); the status line gains the most blocks written in one cycle after `mdt`